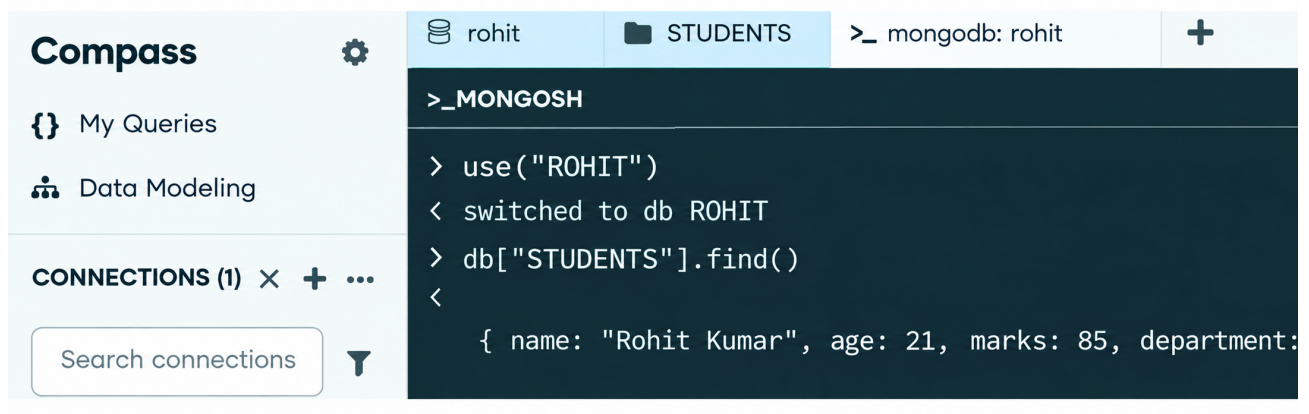


1. CREATE DATABASE

MongoDB Query:

use Students

Explanation: Creates or switches to the Student database.

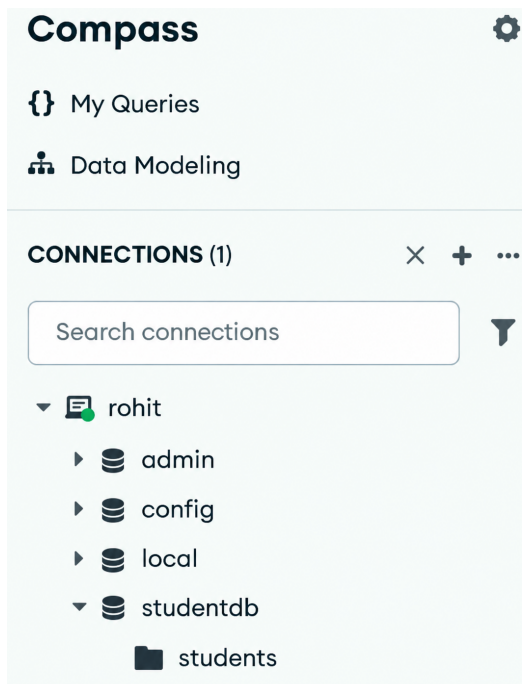


2. CREATE COLLECTION

MongoDB Query:

db.create Collection("ROHIT")

Explanation: Creates a collection named students.



3. INSERT 50 STUDENTS

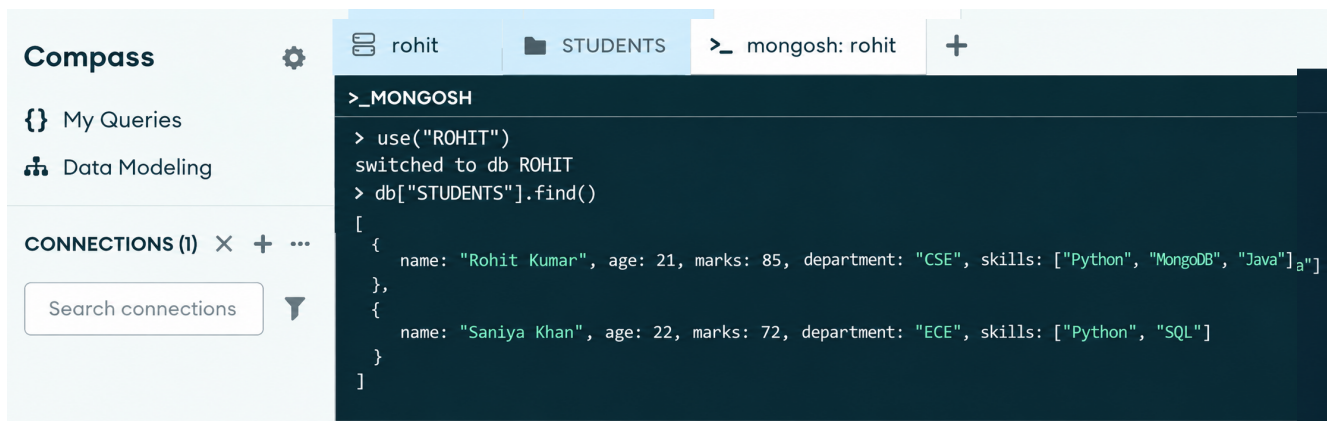
MongoDB Query:

```
db.students.insertMany([  
  
  {roll:1, name:"Aarav", age:18, marks:82, city:"Lucknow"},  
  
  {roll:2, name:"Ishita", age:19, marks:91, city:"Kanpur"},  
  
  {roll:3, name:"Ritvik", age:18, marks:76, city:"Varanasi"},  
  
  {roll:4, name:"Kavya", age:20, marks:88, city:"Ayodhya"},  
  
  {roll:5, name:"Arjun", age:19, marks:79, city:"Prayagraj"},  
  
  {roll:6, name:"Mehak", age:18, marks:94, city:"Lucknow"},  
  
  {roll:7, name:"Dev", age:20, marks:67, city:"Gorakhpur"},  
  
  {roll:8, name:"Riya", age:19, marks:88, city:"Kanpur"},  
  
])
```

{roll:9, name:"Yash", age:18, marks:73, city:"Jaunpur"},
{roll:10, name:"Tanya", age:20, marks:58, city:"Lucknow"},
{roll:11, name:"Samar", age:19, marks:84, city:"Varanasi"},
{roll:12, name:"Nandini", age:18, marks:95, city:"Kanpur"},
{roll:13, name:"Raghav", age:20, marks:71, city:"Ayodhya"},
{roll:14, name:"Muskan", age:19, marks:86, city:"Lucknow"},
{roll:15, name:"Kunal", age:18, marks:63, city:"Prayagraj"},
{roll:16, name:"Simran", age:20, marks:90, city:"Kanpur"},
{roll:17, name:"Devansh", age:19, marks:81, city:"Lucknow"},
{roll:18, name:"Palak", age:18, marks:77, city:"Gonda"},
{roll:19, name:"Aditya", age:20, marks:92, city:"Varanasi"},
{roll:20, name:"Manya", age:19, marks:69, city:"Bareilly"},
{roll:21, name:"Harshit", age:18, marks:87, city:"Lucknow"},
{roll:22, name:"Sakshi", age:20, marks:74, city:"Kanpur"},
{roll:23, name:"Manav", age:19, marks:96, city:"Prayagraj"},
{roll:24, name:"Diya", age:18, marks:83, city:"Ayodhya"},
{roll:25, name:"Rohan", age:20, marks:57, city:"Lucknow"},
{roll:26, name:"Aditi", age:19, marks:89, city:"Varanasi"},
{roll:27, name:"Varun", age:18, marks:72, city:"Gorakhpur"},
{roll:28, name:"Pihu", age:20, marks:85, city:"Kanpur"},
{roll:29, name:"Aniket", age:19, marks:78, city:"Lucknow"},
{roll:30, name:"Ira", age:18, marks:93, city:"Jaunpur"},
{roll:31, name:"Shivam", age:20, marks:66, city:"Ayodhya"},

```
{roll:32, name:"Tanvi", age:19, marks:88, city:"Lucknow"},
{roll:33, name:"Kabir", age:18, marks:54, city:"Kanpur"},
{roll:34, name:"Anjali", age:20, marks:80, city:"Prayagraj"},
{roll:35, name:"Mohit", age:19, marks:91, city:"Varanasi"},
{roll:36, name:"Navya", age:18, marks:70, city:"Lucknow"},
{roll:37, name:"Abhinav", age:20, marks:59, city:"Gonda"},
{roll:38, name:"Sneha", age:19, marks:97, city:"Kanpur"},
{roll:39, name:"Varsha", age:18, marks:75, city:"Lucknow"},
{roll:40, name:"Manish", age:20, marks:68, city:"Bareilly"},
{roll:41, name:"Ansh", age:19, marks:90, city:"Ayodhya"},
{roll:42, name:"Khushi", age:18, marks:86, city:"Lucknow"},
{roll:43, name:"Nikhil", age:20, marks:61, city:"Kanpur"},
{roll:44, name:"Ishaan", age:19, marks:84, city:"Prayagraj"},
{roll:45, name:"Sanya", age:18, marks:73, city:"Varanasi"},
{roll:46, name:"Akash", age:20, marks:87, city:"Lucknow"},
{roll:47, name:"Nisha", age:19, marks:55, city:"Kanpur"},
{roll:48, name:"Reyansh", age:18, marks:92, city:"Jaunpur"},
{roll:49, name:"Shreya", age:20, marks:79, city:"Lucknow"},
{roll:50, name:"Varsha", age:19, marks:88, city:"Kanpur"}
])
```

Explanation: Inserts multiple student documents into the collection.



4. DISPLAY ALL STUDENTS

MongoDB Query:

db. students. Find ()

Explanation: Retrieves documents from the students collection.

```
> db. students. find ()
< [
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca0e'),
    roll: 1,
    name: 'Aarav',
    age: 18,
    marks: 82,
    city: 'Lucknow'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca0f'),
    roll: 2,
    name: 'Ishita',
    age: 19,
    marks: 91,
    city: 'Kanpur'
  }
]
```

5. EQUAL TO — \$eq

```
> db.students.find ({marks: {$eq: 88}})
< [
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca11'),
    roll: 4,
    name: 'Kavya',
    age: 20,
    marks: 88,
    city: 'Ayodhya'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca15'),
    roll: 8,
    name: 'Riya',
    age: 19,
    marks: 88,
    city: 'Kanpur'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca2d'),
    roll: 32,
    name: 'Tanvi',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
]
```

MongoDB Query:

db.students.find ({marks: {\$eq: 88}})

Explanation: Finds students whose marks are exactly 88.

6. GREATER THAN — \$gt

MongoDB Query:

db.students.find({marks: {\$gt: 75}})

```
> db.students.find({marks: {$gt: 75}})
< [
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca0e'),
    roll: 1,
    name: 'Aarav',
    age: 18,
    marks: 82,
    city: 'Lucknow'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca0f'),
    roll: 2,
    name: 'Ishita',
    age: 19,
    marks: 91,
    city: 'Kanpur'
  }
]
```

Explanation: Finds students whose marks are greater than 75.

7. LESS THAN — \$lt

MongoDB Query:

db. students. find ({marks: {\$lt: 58}})

Explanation: Finds students whose marks are less than 58.

```
> db. students. find ({marks: {$lt: 58}})
< {
  _id: ObjectId('6ab6e61c6233435d3a1fca26'),
  roll: 25,
  name: 'Rohan',
  age: 20,
  marks: 57,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab6e61c6233435d3a1fca2e'),
  roll: 33,
  name: 'Kabir',
  age: 18,
  marks: 54,
  city: 'Kanpur'
}
```

8. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

db. students. find({marks: {\$gte: 92}})

```
> db. students. find({marks: {$gte: 92}})
< [
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca13'),
    roll: 6,
    name: 'Mehak',
    age: 18,
    marks: 94,
    city: 'Lucknow'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca19'),
    roll: 12,
    name: 'Nandini',
    age: 18,
    marks: 95,
    city: 'Kanpur'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca20'),
    roll: 19,
    name: 'Aditya',
    age: 20,
    marks: 92,
    city: 'Varanasi'
  }
]
```

Explanation: Finds students whose marks are 92 or above.

9. LESS THAN OR EQUAL TO — \$lte

MongoDB Query:

db. students. find({marks: {\$lte: 67}})

```
> db. students. find({marks: {$lte: 67}})
< {
  _id: ObjectId('6ab6e61c6233435d3a1fca14'),
  roll: 7,
  name: 'Dev',
  age: 20,
  marks: 67,
  city: 'Gorakhpur'
}
{
  _id: ObjectId('6ab6e61c6233435d3a1fca17'),
  roll: 10,
  name: 'Tanya',
  age: 20,
  marks: 58,
  city: 'Lucknow'
}
```

Explanation: Finds students whose marks are 67 or below.

10. UPDATE OPERATION

MongoDB Query:

```
db.students.updateOne(  
  {age:21},  
  {$set:{marks:50}}  
)
```

Explanation: Updates the marks to 50 of the student with age 21.

```
> db.students.updateOne(  
  {age : 21},  
  {$set:{marks:50}}  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

11. DELETE ONE OPERATION

MongoDB Query:

```
> db.students.deleteOne({age : 21})  
< {  
  acknowledged: true,  
  deletedCount: 1  
}
```

db.students.deleteOne({age:21})

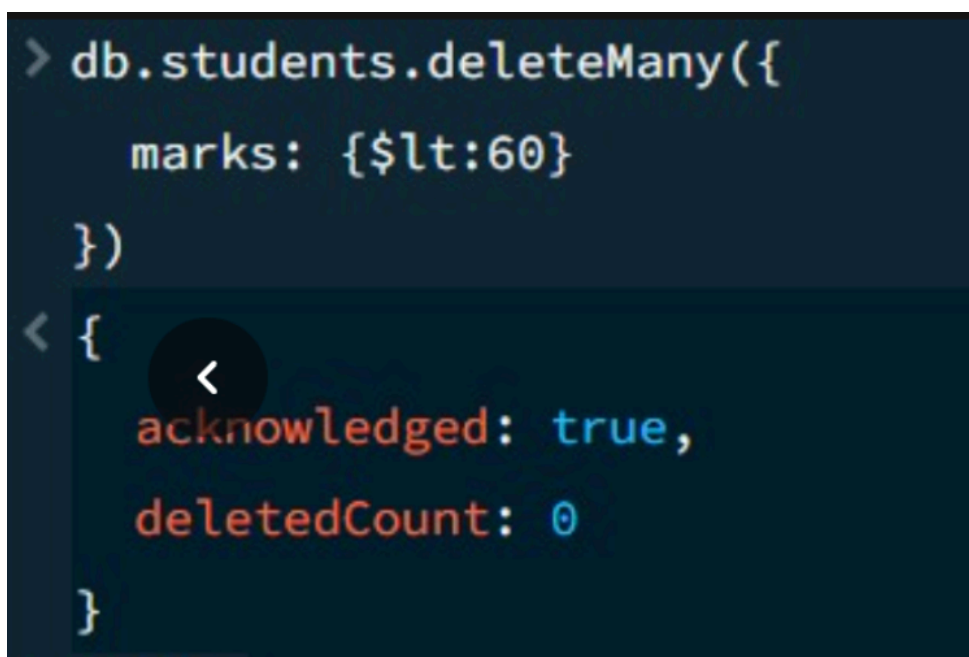
Explanation: Deletes one document matching age 21.

12. DELETE MANY OPERATION

MongoDB Query:

```
db.students.deleteMany({  
  marks: {$lt:60}  
})
```

Explanation: Deletes all students having marks below 60.



```
> db.students.deleteMany({  
  marks: {$lt:60}  
})  
< {  
  acknowledged: true,  
  deletedCount: 0  
}
```

The screenshot shows a MongoDB terminal window with a dark background. The first line shows the command `> db.students.deleteMany({ marks: {$lt:60}})` being entered. The second line shows the response `< { acknowledged: true, deletedCount: 0 }`. A mouse cursor is visible over the response object.

13. AND OPERATION — \$and

MongoDB Query:

```
db.students.find({
  $and:[
    {marks:{$gt:80}},
    {age:19}
  ]
})
```

```
> db.students.find({ $and:[ {marks:{$gt:80}}, {age:19} ] })
< {
  _id: ObjectId('6ab6e61c6233435d3a1fca0f'),
  roll: 2,
  name: 'Ishita',
  age: 19,
  marks: 91,
  city: 'Kanpur'
}
{
  _id: ObjectId('6ab6e61c6233435d3a1fca12'),
  roll: 5,
  name: 'Arjun',
  age: 19,
  marks: 95,
  city: 'Prayagraj'
}
```

Explanation: Finds students satisfying both conditions.

14. OR OPERATION — \$or

MongoDB Query:

```
db.students.find({  
  $or:[  
    {name:"Ayodhya"},  
    {name:"Kanpur"}  
  ]  
})
```

```
> db.students.find([ $or:[ {city:"Ayodhya"}, {city:"Kanpur"} ] ] )  
< [  
  {  
    _id: ObjectId('6ab6e61c6233435d3a1fca0f'),  
    roll: 2,  
    name: 'Ishita',  
    age: 19,  
    marks: 91,  
    city: 'Kanpur'  
  },  
  {  
    _id: ObjectId('6ab6e61c6233435d3a1fca11'),  
    roll: 4,  
    name: 'Kavya',  
    age: 20,  
    marks: 88,  
    city: 'Ayodhya'  
  }  
]
```

```
]  
})
```

Explanation: Finds students name Ayodhya or Kanpur.

15. SORT OPERATION

A.MongoDB Query:

db. students. find (). sort({marks : 1})

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.

```
> db. students. find (). sort({marks : 1})
< [
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca2e'),
    roll: 33,
    name: 'Kabir',
    age: 18,
    marks: 54,
    city: 'Kanpur'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca3c'),
    roll: 47,
    name: 'Nisha',
    age: 19,
    marks: 55,
    city: 'Kanpur'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca26'),
    roll: 25,
    name: 'Rohan',
    age: 20,
    marks: 57,
    city: 'Lucknow'
  }
]
```

B. MongoDB Query:

db. students. find (). Sort ({marks: -1})

```
> db. students. find (). sort ({marks: -1})
< [
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca33'),
    roll: 38,
    name: 'Sneha',
    age: 19,
    marks: 97,
    city: 'Kanpur'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca24'),
    roll: 23,
    name: 'Manav',
    age: 19,
    marks: 96,
    city: 'Prayagraj'
  },
  {

```

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

16. LIMIT OPERATION

MongoDB Query:

db. students. find (). limit (7)

```
> db. students. find (). limit (7)
< [
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca0e'),
    roll: 1,
    name: 'Aarav',
    age: 18,
    marks: 82,
    city: 'Lucknow'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca0f'),
    roll: 2,
    name: 'Ishita',
    age: 19,
    marks: 91,
    city: 'Kanpur'
  },
  {
    _id: ObjectId('6ab6e61c6233435d3a1fca10'),
    roll: 3,
    name: 'Ritvik',
    age: 18,
    marks: 76,
    city: 'Varanasi'
  }
]
```

Explanation: Displays only the first five documents.

17. SEARCH BY CITY

MongoDB Query:

```
db.students.find({age:22})
```

Explanation: Finds all students whose age is 22.

```
> db.students.find({age : 22})
< {
  _id: ObjectId('6ab94b3502beebe9309cf74f'),
  name: 'Saniya Khan',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf753'),
  name: 'Ranveer',
  age: 22,
  marks: 88,
  department: 'ECE',
  skills: [
    'C',
    'Java',
    'SQL'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf758'),
  name: 'Ariun'
```

18. COUNT DOCUMENTS

MongoDB Query:

db. students. count Documents ()

Explanation: Counts documents currently present in the collection (performing after the delete function).

```
> db. students. countDocuments ()  
< 50
```